

TIC TAC TOE in C

✖ Problem Statement:

To design and implement a simple **two-player Tic-Tac-Toe game** in C language. The program should allow two users to play alternately, marking their symbols (X or O) on a 3×3 grid. The game should determine the winner when any player aligns three of their symbols horizontally, vertically, or diagonally, or declare a draw when all cells are filled.

🧠 Concepts Used:

- **Loops:** For iterating through the game board and taking multiple turns.
- **Arrays:** To represent the 3×3 board as a 2D array.
- **Functions:** For modular programming (display board, check winner, make move).
- **Conditionals (if-else):** To check win or draw conditions.
- **Switch Statement:** (Optional) To alternate between player turns.
- **Input/Output:** For reading user moves and displaying the board.

📄 Short Description of Working:

1. The game board is represented using a **3×3-character array** initialized with numbers 1–9.
2. Two players take turns choosing a number corresponding to the board position.
3. The program replaces that number with the player's mark (X or O).
4. After each move, the program checks for a winning combination or draw condition.
5. If a player wins, the program displays the winner's name and ends.
6. If all cells are filled without a winner, it displays *"Game Draw!"*.



tictactoe.c:

```

1  /*-----*/
2      Project Title : Tic Tac Toe Game in C
3      Description   : Two-player game played on a 3x3 grid.
4                      Players take turns marking cells as X or O.
5                      The game ends when one player wins or
6                      when all cells are filled (draw).
7      Concepts Used : Arrays, Loops, Functions, Conditionals, I/O
8  /*-----*/
9
10 #include <stdio.h>
11 #include <conio.h> // Optional: for getch() to pause screen
12 #include <stdlib.h> // For system("cls") to clear screen
13
14 // Global 3x3 array to store the game board
15 char board[3][3] = {
16     {'1', '2', '3'},
17     {'4', '5', '6'},
18     {'7', '8', '9'}
19 };
20
21 // Function Prototypes
22 void displayBoard();
23 int checkWinner();
24 int isDraw();
25 void markBoard(int choice, char mark);
26
27 // Main Function
28 int main() {
29     int player = 1; // Player 1 starts first
30     int choice; // Position selected by player
31     char mark; // X or O
32     int status = -1; // -1 = game continues, 1 = win, 0 = draw
33
34     do {
35         system("cls"); // Clear screen (for better visibility)
36
37         printf("\n\tTIC TAC TOE GAME\n");
38         printf("\t-----\n");
39         printf("\nPlayer 1 (X) - Player 2 (O)\n\n");
40
41         // Display current game board
42         displayBoard();
43
44         // Determine which player's turn it is
45         player = (player % 2) ? 1 : 2;
46         mark = (player == 1) ? 'X' : 'O';
47
48         // Ask the player for a move
49         printf("Player %d, enter the position (1-9): ", player);
50         scanf("%d", &choice);
51
52         // Update board with the player's mark
53         markBoard(choice, mark);
54
55         // Check if someone has won or if it's a draw
56         if (checkWinner()) {
57             system("cls");
58             displayBoard();
59             printf("\n==> Player %d wins! \n", player);
60             status = 1; // Game ends with a win
61             break;
62         } else if (isDraw()) {
63             system("cls");
64             displayBoard();
65             printf("\n==> Game Draw! \n");
66             status = 0; // Game ends with a draw
67             break;
68         }
69
70         player++; // Move to next player
71     } while (status == -1);
72
73     printf("\nThank you for playing Tic Tac Toe!\n");
74
75     getch(); // Wait for key press before closing
76     return 0;
77 }
78

```

```

79
80
81  /*-----
82  Function : displayBoard()
83  Purpose  : Displays the current Tic Tac Toe board
84  -----*/
85 void displayBoard() {
86     printf("\n\t %c | %c | %c ", board[0][0], board[0][1], board[0][2]);
87     printf("\n\t---|---|---");
88     printf("\n\t %c | %c | %c ", board[1][0], board[1][1], board[1][2]);
89     printf("\n\t---|---|---");
90     printf("\n\t %c | %c | %c \n\n", board[2][0], board[2][1], board[2][2]);
91 }
92
93  /*-----
94  Function : markBoard()
95  Purpose  : Places player mark on selected position
96  -----*/
97 void markBoard(int choice, char mark) {
98     int row = (choice - 1) / 3; // Convert 1-9 choice into 2D indices
99     int col = (choice - 1) % 3;
100
101     // Check if position is valid and not already taken
102     if (choice < 1 || choice > 9 || board[row][col] == 'X' || board[row][col] == 'O') {
103         printf("Invalid move! Try again.\n");
104         getch();
105     } else {
106         board[row][col] = mark;
107     }
108 }
109
110  /*-----
111  Function : checkWinner()
112  Purpose  : Checks all possible winning combinations
113  -----*/
114 int checkWinner() {
115     // Check rows and columns
116     for (int i = 0; i < 3; i++) {
117         if ((board[i][0] == board[i][1] && board[i][1] == board[i][2]) ||
118             (board[0][i] == board[1][i] && board[1][i] == board[2][i])) {
119             return 1;
120         }
121     }
122
123     // Check diagonals
124     if ((board[0][0] == board[1][1] && board[1][1] == board[2][2]) ||
125         (board[0][2] == board[1][1] && board[1][1] == board[2][0])) {
126         return 1;
127     }
128
129     return 0; // No winner yet
130 }
131
132  /*-----
133  Function : isDraw()
134  Purpose  : Checks if all cells are filled (draw)
135  -----*/
136 int isDraw() {
137     for (int i = 0; i < 3; i++) {
138         for (int j = 0; j < 3; j++) {
139             if (board[i][j] != 'X' && board[i][j] != 'O')
140                 return 0; // Still empty cell, game continues
141         }
142     }
143     return 1; // All cells filled, game is a draw
144 }

```

Output Screenshot (Representation):

```
TIC TAC TOE GAME
-----
Player 1 (X) - Player 2 (O)

  X | 2 | 3
  ---|---|---
  4 | 0 | X
  ---|---|---
  7 | 8 | 9

Player 2, enter the position (1-9):

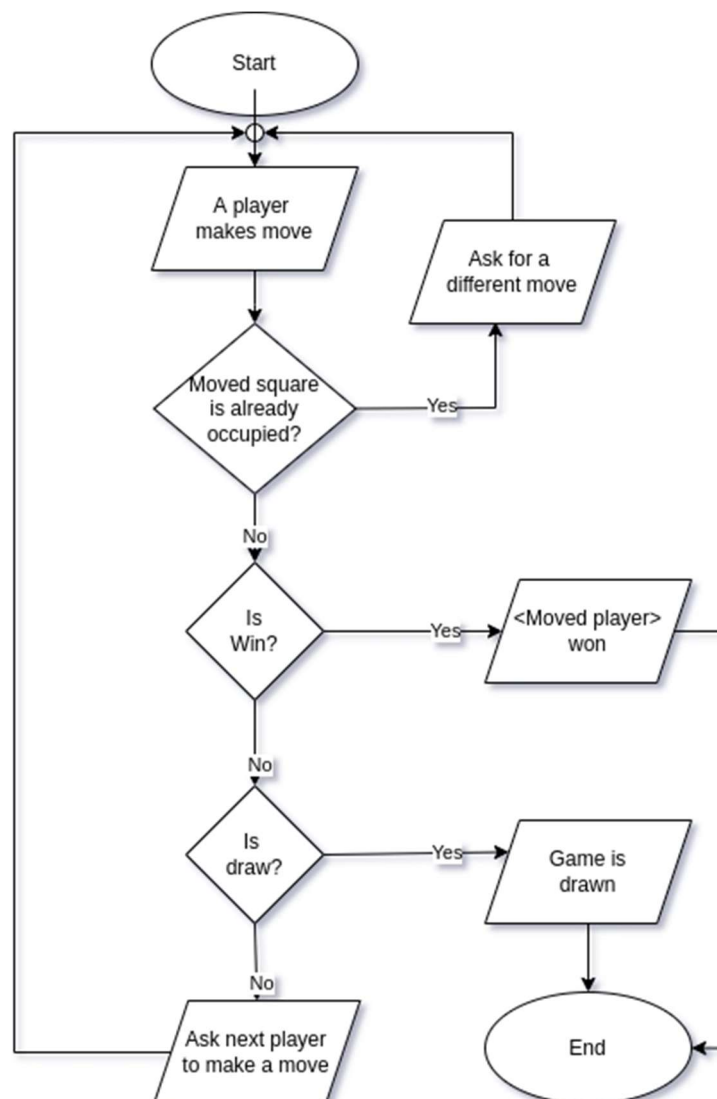
  X | 0 | X
  ---|---|---
  4 | 0 | X
  ---|---|---
  7 | 0 | 9

==> Player 2 wins!
Thank you for playing Tic Tac Toe!
```

🧠 Logic Flow:

1. Start
2. Initialize board with numbers 1–9.
3. Display board.
4. Take input from player 1 → mark X.
5. Check for winner / draw.
6. If no winner → switch to player 2 → mark O.
7. Repeat steps 4–6 until winner or draw.
8. Display result.
9. End.

💡 Flowchart:



Snake and Ladder in C

✖ Problem Statement:

The objective of this project is to design and implement a **Snake and Ladder** game using the C programming language for two players.

Each player rolls a dice and moves across a 10x10 board (from 1 to 100).

The game simulates the movement of players, snakes, and ladders, and determines the winner when a player reaches position **100**.

The project helps understand **loops, conditionals, arrays, random number generation, and function-based modular programming**.

🧠 Concepts Used:

- **Loops:** To iterate through the board and run the game continuously.
- **Arrays:** For the 10x10 conceptual board layout.
- **Functions:** For modularity — separate logic for dice roll, board display, and movement.
- **Conditionals (if-else):** To handle player turns, snakes, ladders, and game outcomes.
- **Random Number Generation (rand()):** To simulate dice rolls between 1 and 6.
- **System Functions (system("cls")):** To refresh and display updated game boards clearly.

📄 Short Description of Working:

1. The program starts with both players at position **0**.
2. On each turn, a player presses a key to roll the dice (a random number from 1–6).
3. The player moves forward by that dice value.
4. If the player lands on a **ladder**, they move **up** to a higher position.
5. If the player lands on a **snake**, they slide **down** to a lower position.
6. The board updates dynamically after every move:
 - **'L'** marks ladder positions.
 - **'S'** marks snake positions.
 - **'P1'** and **'P2'** show the current player positions.
7. The first player to reach **exactly 100** wins the game.
8. The game runs in a loop until a winner is declared or the user exits.



snakeAndLadder.c:

```

1  /*-----
2      Author: Aditya Varma
3      Date: 15/08/2023
4      Project Title   : Snake and Ladder Game
5      Description     : A two-player console-based Snake and Ladder game
6                      : that simulates dice rolls, snakes, ladders, and
7                      : displays the board dynamically with player positions.
8
9      Rules and Features:
10         - The game begins with any dice value (1-6).
11         - If the dice rolls 6, the player gets another chance.
12         - Snakes and Ladders are shown as 'S' and 'L' on the board.
13         - Players are displayed as 'P1' and 'P2' directly on the board.
14         - Snakes: 99→1, 65→40, 25→9
15         - Ladders: 13→42, 60→83, 70→93
16         - The first player to reach exactly 100 wins the game.
17      Concepts Used   : Loops, Functions, Arrays, Conditional Statements,
18                      : Switch Case, Random Number Generation.
19  -----*/
20  #include <stdio.h>
21  #include <stdlib.h>
22
23  /*-----
24      Function: rd
25      Purpose : Generates a random dice value between 1 and 6.
26  -----*/
27  int rd() {
28      int rem;
29      A:
30      rem = rand() % 7;
31      if (rem == 0)
32          goto A;
33      else
34          return rem;
35  }
36
37  /*-----
38      Function: displayChart
39      Purpose : Displays the 10x10 Snake and Ladder board.
40              : Marks Snakes (S), Ladders (L), and Player Positions.
41  -----*/
42  void displayChart(int p1, int p2) {
43      int i, j, num, shift = 0;
44
45      printf("\n\t\t\t\t\t---- SNAKE AND LADDER BOARD ----\n\n");
46
47      // The board contains 10 rows (10x10 grid)
48      for (i = 10; i > 0; i--) {
49          int rowBase = (i - 1) * 10;
50
51          // Left-to-right numbering
52          if (shift % 2 == 0) {
53              for (j = 10; j >= 1; j--) {
54                  num = rowBase + j;
55
56                  // Display player positions
57                  if (num == p1 && num == p2)
58                      printf("P1P2\t");
59                  else if (num == p1)
60                      printf("P1\t");
61                  else if (num == p2)
62                      printf("P2\t");
63
64                  // Display Snakes and Ladders
65                  else if (num == 99 || num == 65 || num == 25)
66                      printf("S\t");
67                  else if (num == 70 || num == 60 || num == 13)
68                      printf("L\t");
69                  else
70                      printf("%d\t", num);
71              }
72          } else { // Right-to-left numbering
73              for (j = 1; j <= 10; j++) {
74                  num = rowBase + j;

```

```

76         if (num == p1 && num == p2)
77             printf("P1P2\t");
78         else if (num == p1)
79             printf("P1\t");
80         else if (num == p2)
81             printf("P2\t");
82         else if (num == 99 || num == 65 || num == 25)
83             printf("S\t");
84         else if (num == 70 || num == 60 || num == 13)
85             printf("L\t");
86         else
87             printf("%d\t", num);
88     }
89 }
90
91 shift++;
92 printf("\n\n");
93 }
94
95 printf("-----\n");
96 printf("Player 1 Position: %d\t\t\tPlayer 2 Position: %d\n", p1, p2);
97 printf("-----\n");
98 }
99
100 /*-----
101  Function: movePlayer
102  Purpose : Moves the player according to dice value and applies
103           Snake or Ladder effect if present.
104  -----*/
105 int movePlayer(int curPos, int dice, int playerNum) {
106     curPos += dice;
107
108     // Check if player exceeds 100
109     if (curPos > 100) {
110         curPos -= dice;
111         printf("Player %d exceeded 100! Staying at %d.\n", playerNum, curPos);
112         return curPos;
113     }
114
115     // Snake positions
116     if
117         (curPos == 99) curPos = 1;
118     else if
119         (curPos == 65) curPos = 40;
120     else if
121         (curPos == 25) curPos = 9;
122
123     // Ladder positions
124     else if
125         (curPos == 13) curPos = 42;
126     else if
127         (curPos == 60) curPos = 83;
128     else if
129         (curPos == 70) curPos = 93;
130
131     return curPos;
132 }
133
134 /*-----
135  Function: main
136  Purpose : The main control function to execute the game.
137           Handles player turns, dice rolls, and win conditions.
138  -----*/
139 int main() {
140     int curPos1 = 0, curPos2 = 0, dice;
141     char choice;
142
143     while (1) {
144         system("cls"); // Clears the screen (Windows only)
145         printf("\n\t\t\t\t\t** SNAKE AND LADDER GAME **\n\t\t\t\t\t(Two Player Version)\n");
146         printf("-----\n");

```

```

147
148     displayChart(curPos1, curPos2);
149
150     printf("\n1. Player 1 plays\n2. Player 2 plays\n3. Exit\nEnter your choice: ");
151     scanf(" %c", &choice);
152
153     switch (choice) {
154         case '1':
155             dice = rd();
156             printf("\nPlayer 1 rolled a %d\n", dice);
157             curPos1 = movePlayer(curPos1, dice, 1);
158
159             if (curPos1 == 100) {
160                 displayChart(curPos1, curPos2);
161                 printf("\n Player 1 Wins the Game! \n");
162                 exit(0);
163             }
164             break;
165
166         case '2':
167             dice = rd();
168             printf("\nPlayer 2 rolled a %d\n", dice);
169             curPos2 = movePlayer(curPos2, dice, 2);
170
171             if (curPos2 == 100) {
172                 displayChart(curPos1, curPos2);
173                 printf("\n Player 2 Wins the Game! \n");
174                 exit(0);
175             }
176             break;
177
178         case '3':
179             printf("Thanks for playing!\n");
180             exit(0);
181
182         default:
183             printf("Invalid choice! Try again.\n");
184     }
185
186     printf("\nPress Enter to continue...");
187     getchar();
188     getchar();
189 }
190
191 return 0;
192 }

```

Output Screenshot (Representation):

```

** SNAKE AND LADDER GAME **
(Two Player Version)

-----
---- SNAKE AND LADDER BOARD ----
100  S   98  97  96  95  94  93  92  91
81   82  83  84  85  86  87  88  89  90
80   79  78  77  76  75  74  73  72  71
61   62  63  64  S   66  67  68  69  L
L   59  58  57  56  55  54  53  52  51
41   42  43  44  45  46  47  48  49  50
40   39  38  37  36  35  34  33  32  31
21   22  23  24  S   26  27  28  29  30
20   P1  18  17  16  15  P2  L   12  11
1    2   3   4   5   6   7   8   9  10

-----
Player 1 Position: 19 | Player 2 Position: 14
-----

1. Player 1 plays
2. Player 2 plays
3. Exit
Enter your choice:

```

💡 Flowchart:

Snakes and Ladders Game

